Embedded Post-Quantum Cryptography & Hardware Security

Comprehensive Experimental Benchmark Report & Codebase Walkthrough

Author: Zain Ul Haq — **Program:** M.Sc. Autonomous Systems (H-BRS / b-it)
Supervisors: Prof. Dr. Sebastian Houben, Prof. Dr. Hannes Tschofenig
Date: September 2026 — **Hardware Testbeds:** NXP FRDM-MCXN947 (Cortex-M33) & ESP32-S3 (Xtensa LX7)

Executive Overview & Research Scope

This report delivers a unified, self-contained, and mathematically rigorous specification of the entire embedded systems, security, and post-quantum cryptographic (PQC) benchmarking suite developed for this Master's Thesis. Per experimental design requirements, all machine learning, neural network, and conformal prediction workflows have been completely decoupled, focusing strictly on **pure embedded systems, microarchitectural profiling, protocol energy modeling, and real-time scheduling**.

Every metric, latency figure, stack memory high-water mark, and daily energy curve reported herein is 100% grounded in physical hardware-in-the-loop datasets and reproducible using the provided codebase.

1 EEMBC-Style Benchmark Methodology & Primitive Mappings

To establish an authoritative, industry-aligned baseline, our experiments adapt the methodology of the **Embedded Microprocessor Benchmark Consortium (EEMBC)**—specifically the EEMBC SecureMark-TLS and ULPMark standards—to evaluate NIST-standardized Post-Quantum Cryptography.

1.1 Primitive-to-Primitive Protocol Handshake Mapping

To provide an apples-to-apples comparison between classical TLS 1.2 cryptography and quantum-safe algorithms, we evaluate equivalent authentication and key exchange handshake phases:

- **Classical Baseline Phase (EEMBC SecureMark-TLS):** Measures mbedTLS executing 1× ECDH P-256 scalar multiplication + 1× ECDSA P-256 signature generation + 2× ECDSA P-256 signature verifications.
- **Post-Quantum Equivalent Phase (PQClean Reference Suite):** Measures 1× ML-KEM key exchange (Keypair + Encapsulation + Decapsulation) + 1× ML-DSA digital signature generation + 2× ML-DSA signature verifications.
- **Security Equivalence Levels:** Classical 128-bit security (P-256 / AES-128) is directly compared against **NIST PQC Security Level 1** (ML-KEM-512 + ML-DSA-44), **Level 3** (ML-KEM-768 + ML-DSA-65), and **Level 5** (ML-KEM-1024 + ML-DSA-87).

1.2 Physical Measurement Instrumentation

- **Clock Cycles & Execution Latency:** Captured natively via internal hardware registers—ARM DWT `CYCCNT` on the NXP FRDM-MCXN947 (150 MHz) and Xtensa `CCOUNT` on the ESP32-S3 (240 MHz). Tests run free-running with debuggers and UART streams detached to eliminate clock pollution.
- **Peak Stack Memory (RAM High-Water Mark):** Thread stack regions are painted with a distinct `0xEE` byte pattern at boot. Post-execution memory scans inspect the unwritten boundary via `k_thread_stack_space_get` to record exact peak dynamic memory usage.
- **High-Frequency Active Energy:** Physical active currents are measured in real time using a Nordic Power Profiler Kit II (PPK2) sampling at **10 kHz** in series shunt mode across the board's 3.3V power rails.

2 Bare-Metal EEMBC SecureMark-TLS Baseline (Classical)

The classical baseline was established by compiling and executing the official self-hosted EEMBC SecureMark-TLS benchmark suite across 14 component cryptographic operations (AES-128-ECB/CCM, SHA-256, ECDH P-256, ECDSA P-256).

- **NXP FRDM-MCXN947 (Cortex-M33 @ 150 MHz, `-O2`):** Achieved an official validated score of **5,472.08 SecureMark Score** (Passed 14/14 component CRC checks; verified via `securemark_score.txt`).
- **ESP32-S3 DevKitC (Xtensa LX7 @ 240 MHz, `-Os`):** Achieved an official score of **1,895.06 SecureMark Score** (Passed 14/14 component CRC checks; verified via `securemark_esp32s3_score.txt`).

Low-Level Porting Fixes Applied: 1) *ECP Point Blinding PRNG Fix:* Implemented a lightweight `xorshift` PRNG to resolve null-pointer `MBEDTLS_ERR_ECP_BAD_INPUT_DATA` aborts during scalar multiplication; 2) *SEC1 Format Conversion:* Prepended the SEC1 `0x04` byte to raw 64-byte `X||Y` EEMBC public keys; 3) *ASN.1 DER Signature Formatting:* Encoded raw `r||s` signature registers into standard DER structures for EEMBC CRC validation.

3 Microarchitectural Post-Quantum Benchmarks (EEMBC-Style)

Using PQClean reference implementations compiled for bare-metal execution, we benchmarked NIST FIPS-203 ML-KEM and FIPS-204 ML-DSA primitives.

Table 1: EEMBC-Style Microarchitectural PQC Performance Profile (NXP FRDM-MCXN947 @ 150 MHz)

Algorithm	Operation	Mean Cycles	Latency (ms)	Stack HWM	Flash Text	Static Data
ML-KEM-512	Keypair	2,978,432	19.86 ms	8,960 B	41,648 B	40,928 B
ML-KEM-512	Encapsulate	3,503,918	23.36 ms	11,592 B	41,648 B	40,928 B
ML-KEM-512	Decapsulate	4,342,471	28.95 ms	12,344 B	41,648 B	40,928 B
ML-KEM-768	Keypair	4,893,951	32.63 ms	13,056 B	41,432 B	42,400 B
ML-KEM-768	Encapsulate	5,692,464	37.95 ms	16,200 B	41,432 B	42,400 B
ML-KEM-768	Decapsulate	6,809,160	45.39 ms	17,272 B	41,432 B	42,400 B
ML-KEM-1024	Keypair	7,644,704	50.96 ms	18,176 B	41,944 B	44,032 B
ML-KEM-1024	Encapsulate	8,582,539	57.22 ms	21,832 B	41,944 B	44,032 B
ML-KEM-1024	Decapsulate	10,008,989	66.73 ms	23,384 B	41,944 B	44,032 B
ML-DSA-87	Sign (Median)	18,790,358	125.27 ms	28,117 B	44,000 B	42,100 B
ML-DSA-87	Sign (Worst-Case)	90,540,000	603.60 ms	28,117 B	44,000 B	42,100 B

Key Empirical Findings:

- **Peak Stack RAM Efficiency:** ML-KEM-768 decapsulation requires **17,272 bytes** of peak dynamic stack RAM, proving that post-quantum key exchange easily fits within modern microcontroller SRAM (512 KB) without memory overflow.
- **Algorithmic Timing Fat-Tails:** FIPS-204 ML-DSA digital signing exhibits a massive worst-to-median latency spike of **4.8× to 8.1×** (up to 603.6 ms) caused by Fiat-Shamir rejection sampling. Because retry counts per seed are byte-for-byte identical across Cortex-M33 and Xtensa silicon, this timing jitter is **purely algorithmic** and cannot be compiled away.

4 Classical vs. Post-Quantum TLS 1.2 Handshake Migration Study

By combining baseline EEMBC SecureMark-TLS timings with bare-metal PQC measurements, we evaluated the net performance impact of migrating an embedded TLS handshake from classical ECDH/ECDSA to NIST Post-Quantum Cryptography.

Table 2: Handshake Migration Performance & Energy Comparison ($1 \times \text{KEX} + 1 \times \text{Sign} + 2 \times \text{Verify}$)

Target Platform	Cryptographic Suite	Handshake Latency	Latency Delta	Active Energy
NXP MCXN947	Classical (ECDH + ECDSA P-256)	166.10 ms	Baseline	1.36 m
NXP MCXN947	PQC Level 1 (ML-KEM-512 + ML-DSA-44)	151.00 ms	**−9.1% (Faster)**	1.24 m
NXP MCXN947	PQC Level 3 (ML-KEM-768 + ML-DSA-65)	250.60 ms	+50.9%	2.05 m
NXP MCXN947	PQC Level 5 (ML-KEM-1024 + ML-DSA-87)	375.20 ms	+125.9%	3.08 m
ESP32-S3	Classical (ECDH + ECDSA P-256)	495.20 ms	Baseline	48.30 m
ESP32-S3	PQC Level 1 (ML-KEM-512 + ML-DSA-44)	88.80 ms	**−82.1% (Faster)**	8.68 m
ESP32-S3	PQC Level 3 (ML-KEM-768 + ML-DSA-65)	134.20 ms	**−72.9% (Faster)**	13.12 m
ESP32-S3	PQC Level 5 (ML-KEM-1024 + ML-DSA-87)	187.10 ms	**−62.2% (Faster)**	18.28 m

Takeaway: Migrating to NIST Security Level 1 PQC is actually **9.1% faster on the NXP board and 82.1% faster on the ESP32-S3** than classical ECDH/ECDSA. This occurs because asymmetric ML-DSA signature verification is dramatically cheaper computationally than classical ECDSA verification.

5 DTLS Connection ID (RFC 9146) Energy Amortization

Executing a full post-quantum handshake on every device wakeup severely degrades coin-cell battery life. To mitigate this, we evaluated a 24-hour battery deployment model utilizing **DTLS Connection IDs (CIDs)** for lightweight symmetric session resumption.

Table 3: 24-Hour Battery Energy Model on CR2032 Coin Cell (225 mAh) across Wakeup Duty Cycles

Board Target	Cryptographic Suite	Wakeups / Day	Full Handshake Energy	DTLS CID Energy	Daily Savi
MCXN947	PQC (768 + 87)	24 / day	1,340.91 mJ	1,297.90 mJ	3.2
MCXN947	PQC (768 + 87)	288 / day	1,834.92 mJ	1,298.23 mJ	29.2
MCXN947	PQC (768 + 87)	2,880 / day	6,685.23 mJ	1,301.50 mJ	**80.5**
ESP32-S3	PQC (768 + 87)	24 / day	1,735.13 mJ	1,315.61 mJ	24.1
ESP32-S3	PQC (768 + 87)	288 / day	6,565.57 mJ	1,330.69 mJ	79.7
ESP32-S3	PQC (768 + 87)	2,880 / day	53,991.65 mJ	1,478.69 mJ	**97.2**

Battery Longevity Impact: Under dense duty cycles (2,880 wakeups/day), DTLS CID session resumption extends coin-cell battery life from **88 days to 1,740 days (a 19× battery life extension)**.

6 Sliced Cooperative Scheduling & Side-Channel Obfuscation

To prevent heavy PQC handshakes from starving real-time application loops on a single CPU core, we benchmarked a **Single-Core Cooperative Sliced Scheduler**.

- **Deadline Miss Elimination:** Slicing handshakes into non-blocking chunks executed during frame timing slack reduces real-time 30 FPS deadline misses from **10.0% down to 0.0%** on a single core.
- **Execution Footprint Reduction:** Cooperative interleaving reduces combined CPU execution time by **14% to 15%** compared to sequential blocking.
- **Side-Channel Power Leakage Defense:** To counteract activity leakage over power rails, we implemented probabilistic dummy-state insertion. Injecting dummy operations ($p = 0.3$) into idle slices degrades an attacker's side-channel classification accuracy from 100% down to **75.5%**.

7 Human-Readable Codebase Architecture & Reproducibility Guide

This section provides a clear, natural walkthrough of how the repository is structured, where each experiment lives, and how to execute the code to reproduce all reported findings.

7.1 Repository Folder Structure

- `codebase/projects/securemark_tls_mcxn947/`: NXP MCXN947 mbedTLS integration, LinkServer scripts, and `migration_compare.py`.
- `codebase/projects/securemark_tls_esp32s3/`: ESP32-S3 ESP-IDF port, `build.sh` scripts, and `cross_platform_compa`
- `codebase/mlkem_bench/` & `mldsa_bench_nxp/`: Bare-metal PQC drivers, hardware cycle counting (`dwt.h`), and stack painting modules.
- `codebase/securemark_tls_mcxn947/dtls_cid_model.py`: 24-hour coin-cell battery longevity simulation script.
- `knowledge/`: Contains all pre-collected physical hardware CSV datasets (`dtls_cid_daily_energy.csv`, `mlkem_perf_memor` `mldsa87_sign_raw.csv`).

7.2 Execution Environment Boundaries

- **Requires Target Hardware:** Flashing and running SecureMark ports or raw cycle/stack captures requires physical silicon (MCXN947 or ESP32-S3).
- **Runs Offline on Any PC:** All migration analysis scripts, DTLS CID battery models, and summary verifications execute purely in Python from the repository root using the bundled CSV datasets.

7.3 Step-by-Step Execution Commands

Run all commands from the repository root directory:

1. **Reproduce MCXN947 Migration Analysis (Section 4):**

   ```
   python codebase/projects/securemark_tls_mcxn947/migration_compare.py
   ```

   Expected Output: Generates `results/MIGRATION_COMPARISON.md` confirming Level 1 PQC is −9.1% faster than classical ECDH/ECDSA.

2. **Reproduce ESP32-S3 Cross-Platform Migration (Section 4):**

   ```
   python codebase/projects/securemark_tls_esp32s3/cross_platform_compare.py
   ```

   Expected Output: Generates `results/CROSS_PLATFORM_MIGRATION.md` showing latency drops of −82.1% (Level 1) and −72.9% (Level 3).

3. **Reproduce DTLS CID 24-Hour Battery Model (Section 5):**

   ```
   python codebase/projects/securemark_tls_mcxn947/dtls_cid_model.py
   ```

   Expected Output: Computes daily energy profiles and writes `results/dtls_cid/daily_energy.csv`, confirming up to 97.26% battery savings (19× lifespan extension).

4. **Build & Flash ESP32-S3 SecureMark-TLS (Section 2):**

   ```
   bash codebase/projects/securemark_tls_esp32s3/scripts/build.sh build
   bash codebase/projects/securemark_tls_esp32s3/scripts/build.sh -p COM17 flash
   ```

   Expected Output: Serial terminal outputs a validated **1,895 SecureMark Score** with 14/14 component CRC passes.